interface (SBI). An SBI comprises the user-level and supervisor-level ISA together with a set of SBI function calls. Using a single SBI across all SEE implementations allows a single OS binary image to run on any SEE. The SEE can be a simple boot loader and BIOS-style IO system in a low-end hardware platform, or a hypervisor-provided virtual machine in a high-end server, or a thin translation layer over a host operating system in an architecture simulation environment.



Most supervisor-level ISA definitions do not separate the SBI from the execution environment and/or the hardware platform, complicating virtualization and bring-up of new hardware platforms.

The rightmost configuration shows a virtual machine monitor configuration where multiple multiprogrammed OSs are supported by a single hypervisor. Each OS communicates via an SBI with the hypervisor, which provides the SEE. The hypervisor communicates with the hypervisor execution environment (HEE) using a hypervisor binary interface (HBI), to isolate the hypervisor from details of the hardware platform.



The ABI, SBI, and HBI are still a work-in-progress, but we are now prioritizing support for Type-2 hypervisors where the SBI is provided recursively by an S-mode OS.

Hardware implementations of the RISC-V ISA will generally require additional features beyond the privileged ISA to support the various execution environments (AEE, SEE, or HEE).

1.2. Privilege Levels

At any time, a RISC-V hardware thread (*hart*) is running at some privilege level encoded as a mode in one or more CSRs (control and status registers). Three RISC-V privilege levels are currently defined as shown in Table 1.

Table 1. RISC-V privilege levels.

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	*Reserved*	
3	11	Machine	M

Privilege levels are used to provide protection between different components of the software stack, and attempts to perform operations not permitted by the current privilege mode will cause an exception to be raised. These exceptions will normally cause traps into an underlying execution environment.



In the description, we try to separate the privilege level for which code is written, from the privilege mode in which it runs, although the two are often tied. For example, a supervisor-level operating system can run in supervisor-mode on a system with three privilege modes, but can also run in user-mode under a classic virtual machine monitor on systems with two or more privilege modes. In both cases, the same supervisor-level operating system binary code can be used, coded to a supervisor-level SBI and hence expecting to be able to use supervisor-level privileged instructions and CSRs. When running a guest OS in user mode, all supervisor-level actions will be trapped and emulated by the SEE running in the higher-privilege level.

The machine level has the highest privileges and is the only mandatory privilege level for a RISC-V hardware platform. Code run in machine-mode (M-mode) is usually inherently trusted, as it has low-level access to the machine implementation. M-mode can be used to manage secure execution environments on RISC-V. User-mode (U-mode) and supervisor-mode (S-mode) are intended for conventional application and operating system usage respectively.

Each privilege level has a core set of privileged ISA extensions with optional extensions and variants. For example, machine-mode supports an optional standard extension for memory protection. Also, supervisor mode can be extended to support Type-2 hypervisor execution as described in Chapter 21.

Implementations might provide anywhere from 1 to 3 privilege modes trading off reduced isolation for lower implementation cost, as shown in Table 2.

Table 2. Supported combination of privilege modes.

Number of levels	Supported Modes	Intended Usage
1	M	Simple embedded systems
2	M, U	Secure embedded systems
3	M, S, U	Systems running Unix-like operating systems

All hardware implementations must provide M-mode, as this is the only mode that has unfettered access to the whole machine. The simplest RISC-V implementations may provide only M-mode, though this will provide no protection against incorrect or malicious application code.



The lock feature of the optional PMP facility can provide some limited protection even with only M-mode implemented.

Many RISC-V implementations will also support at least user mode (U-mode) to protect the rest of the system from application code. Supervisor mode (S-mode) can be added to provide isolation between a supervisor-level operating system and the SEE.

A hart normally runs application code in U-mode until some trap (e.g., a supervisor call or a timer interrupt) forces a switch to a trap handler, which usually runs in a more privileged mode. The hart will then execute the trap handler, which will eventually resume execution at or after the original trapped instruction in U-mode. Traps that increase privilege level are termed *vertical* traps, while traps that remain at the same privilege level are termed *horizontal* traps. The RISC-V privileged architecture provides flexible routing of traps to different privilege layers.



Horizontal traps can be implemented as vertical traps that return control to a horizontal trap handler in the less-privileged mode.

1.3. Debug Mode

Implementations may also include a debug mode to support off-chip debugging and/or manufacturing test. Debug mode (D-mode) can be considered an additional privilege mode, with even more access than M-mode. The separate debug specification proposal describes operation of a RISC-V hart in debug mode. Debug mode reserves a few CSR addresses that are only accessible in D-mode, and may also reserve some portions of the physical address space on a platform.

higher privilege level. This avoids trapping permitted lower-privilege accesses while still causing traps on illegal accesses. Currently, the counters are the only shadowed CSRs.

2.2. CSR Listing

Table 4-Table 8 list the CSRs that have currently been allocated CSR addresses. The timers, counters, and floating-point CSRs are standard unprivileged CSRs. The other registers are used by privileged code, as described in the following chapters. Note that not all registers are required on all implementations.

Table 3. Allocation of RISC-V CSR address ranges.

CSR Address			Hex	Use and Accessibility
[11:10]	[9:8]	[7:4]		
Unprivileged and User-Level CSRs				
00	00	XXXX	0x000-0x0FF	Standard read/write
01	00	XXXX	0x400-0x4FF	Standard read/write
10	00	XXXX	0x800-0x8FF	Custom read/write
11	00	0XXX	0xC00-0xC7F	Standard read-only
11	00	10XX	0xC80-0xCBF	Standard read-only
11	00	11XX	0xCC0-0xCFF	Custom read-only
Supervisor-Level CSRs				
00	01	XXXX	0x100-0x1FF	Standard read/write
01	01	0XXX	0x500-0x57F	Standard read/write
01	01	10XX	0x580-0x5BF	Standard read/write
01	01	11XX	0x5C0-0x5FF	Custom read/write
10	01	0XXX	0x900-0x97F	Standard read/write
10	01	10XX	0x980-0x9BF	Standard read/write
10	01	11XX	0x9C0-0x9FF	Custom read/write
11	01	0XXX	0xD00-0xD7F	Standard read-only
11	01	10XX	0xD80-0xDBF	Standard read-only
11	01	11XX	0xDC0-0xDFF	Custom read-only
Hypervisor and VS CSRs				
00	10	XXXX	0x200-0x2FF	Standard read/write
01	10	0XXX	0x600-0x67F	Standard read/write
01	10	10XX	0x680-0x6BF	Standard read/write
01	10	11XX	0x6C0-0x6FF	Custom read/write
10	10	0XXX	0xA00-0xA7F	Standard read/write
10	10	10XX	0xA80-0xABF	Standard read/write

10	10	11XX	0xAC0-0xAFF	Custom read/write
11	10	0XXX	0xE00-0xE7F	Standard read-only
11	10	10XX	0xE80-0xEBF	Standard read-only
11	10	11XX	0xEC0-0xEFF	Custom read-only
Machine-Level CSRs				
00	11	XXXX	0x300-0x3FF	Standard read/write
01	11	0XXX	0x700-0x77F	Standard read/write
01	11	100X	0x780-0x79F	Standard read/write
01	11	1010	0x7A0-0x7AF	Standard read/write debug CSRs
01	11	1011	0x7B0-0x7BF	Debug-mode-only CSRs
01	11	11XX	0x7C0-0x7FF	Custom read/write
10	11	0XXX	0xB00-0xB7F	Standard read/write
10	11	10XX	0xB80-0xBBF	Standard read/write
10	11	11XX	0xBC0-0xBFF	Custom read/write
11	11	0XXX	0xF00-0xF7F	Standard read-only
11	11	10XX	0xF80-0xFBF	Standard read-only
11	11	11XX	0xFC0-0xFFF	Custom read-only

Table 4. Currently allocated RISC-V unprivileged CSR addresses.

Number	Privilege	Name	Description
Unprivileged Floating-Point CSRs			
0x001	URW	fflags	Floating-Point Accrued Exceptions.
0x002	URW	frm	Floating-Point Dynamic Rounding Mode.
0x003	URW	fcsr	Floating-Point Control and Status Register (frm +fflags).
Unprivileged Vector CSRs			
0x008	URW	vstart	Vector start position.
0x009	URW	vxsat	Fixed-point accrued saturation flag.
0x00A	URW	vxrm	Fixed-point rounding mode.
0x00F	URW	vcsr	Vector control and status register.
0xC20	URO	vl	Vector length.
0xC21	URO	vtype	Vector data type register.
0xC22	URO	vlenb	Vector register length in bytes.
Unprivileged Zicfiss extension CSR			
0x011	URW	ssp	Shadow Stack Pointer.
Unprivileged Entropy Source Extension CSR			
0x015	URW	seed	Seed for cryptographic random bit generators.
Unprivileged Zcmt Extension CSR			
0x017	URW	jvt	Table jump base vector and control register.
Unprivileged Counter/Timers			
0xC00	URO	cycle	Cycle counter for RDCYCLE instruction.
0xC01	URO	time	Timer for RDTIME instruction.
0xC02	URO	instret	Instructions-retired counter for RDINSTRET instruction.
0xC03	URO	hpmcounter3	Performance-monitoring counter.
0xC04	URO	hpmcounter4	Performance-monitoring counter.
		⋮	
0xC1F	URO	hpmcounter31	Performance-monitoring counter.
0xC80	URO	cycleh	Upper 32 bits of cycle, RV32 only.
0xC81	URO	timeh	Upper 32 bits of time, RV32 only.
0xC82	URO	instreth	Upper 32 bits of instret, RV32 only.
0xC83	URO	hpmcounter3h	Upper 32 bits of hpmcounter3, RV32 only.
0xC84	URO	hpmcounter4h	Upper 32 bits of hpmcounter4, RV32 only.
		⋮	
0xC9F	URO	hpmcounter31h	Upper 32 bits of hpmcounter31, RV32 only.

Table 5. Currently allocated RISC-V supervisor-level CSR addresses.

Number	Privilege	Name	Description
Supervisor Trap Setup			
0x100	SRW	sstatus	Supervisor status register.
0x104	SRW	sie	Supervisor interrupt-enable register.
0x105	SRW	stvec	Supervisor trap handler base address.
0x106	SRW	scounteren	Supervisor counter enable.
Supervisor Configuration			
0x10A	SRW	senvcfg	Supervisor environment configuration register.
Supervisor Counter Setup			
0x120	SRW	scountinhibit	Supervisor counter-inhibit register.
Supervisor Trap Handling			
0x140	SRW	sscratch	Supervisor scratch register.
0x141	SRW	sepc	Supervisor exception program counter.
0x142	SRW	scause	Supervisor trap cause.
0x143	SRW	stval	Supervisor trap value.
0x144	SRW	sip	Supervisor interrupt pending.
0xDA0	SRO	scountovf	Supervisor count overflow.
Supervisor Protection and Translation			
0x180	SRW	satp	Supervisor address translation and protection.
Debug/Trace Registers			
0x5A8	SRW	scontext	Supervisor-mode context register.
Supervisor State Enable Registers			
0x10C	SRW	sstateen0	Supervisor State Enable 0 Register.
0x10D	SRW	sstateen1	Supervisor State Enable 1 Register.
0x10E	SRW	sstateen2	Supervisor State Enable 2 Register.
0x10F	SRW	sstateen3	Supervisor State Enable 3 Register.

Table 6. Currently allocated RISC-V hypervisor and VS CSR addresses.

Number	Privilege	Name	Description
		Hypervisor Trap Setup	
0x600	HRW	hstatus	Hypervisor status register.
0x602	HRW	hedeleg	Hypervisor exception delegation register.
0x603	HRW	hideleg	Hypervisor interrupt delegation register.
0x604	HRW	hie	Hypervisor interrupt-enable register.
0x606	HRW	hcounteren	Hypervisor counter enable.
0x607	HRW	hgeie	Hypervisor guest external interrupt-enable register.
0x612	HRW	hedelegh	Upper 32 bits of hedeleg, RV32 only.
		Hypervisor Trap Handling	
0x643	HRW	htval	Hypervisor trap value.
0x644	HRW	hip	Hypervisor interrupt pending.
0x645	HRW	hvip	Hypervisor virtual interrupt pending.
0x64A	HRW	htinst	Hypervisor trap instruction (transformed).
0xE12	HRO	hgeip	Hypervisor guest external interrupt pending.
		Hypervisor Configuration	
0x60A	HRW	henvcfg	Hypervisor environment configuration register.
0x61A	HRW	henvcfgh	Upper 32 bits of henvcfg, RV32 only.
		Hypervisor Protection and Translation	
0x680	HRW	hgatp	Hypervisor guest address translation and protection.
		Debug/Trace Registers	
0x6A8	HRW	hcontext	Hypervisor-mode context register.
		Hypervisor Counter/Timer Virtualization Registers	
0x605	HRW	htimedelta	Delta for VS/VU-mode timer.
0x615	HRW	htimedeltah	Upper 32 bits of htimedelta, RV32 only.
		Hypervisor State Enable Registers	
0x60C	HRW	hstateen0	Hypervisor State Enable 0 Register.
0x60D	HRW	hstateen1	Hypervisor State Enable 1 Register.
0x60E	HRW	hstateen2	Hypervisor State Enable 2 Register.
0x60F	HRW	hstateen3	Hypervisor State Enable 3 Register.
0x61C	HRW	hstateen0h	Upper 32 bits of Hypervisor State Enable 0 Register, RV32 only.
0x61D	HRW	hstateen1h	Upper 32 bits of Hypervisor State Enable 1 Register, RV32 only.
0x61E	HRW	hstateen2h	Upper 32 bits of Hypervisor State Enable 2 Register, RV32 only.
0x61F	HRW	hstateen3h	Upper 32 bits of Hypervisor State Enable 3 Register, RV32 only.
		Virtual Supervisor Registers	
0x200	HRW	vsstatus	Virtual supervisor status register.
0x204	HRW	vsie	Virtual supervisor interrupt-enable register.
0x205	HRW	vstvec	Virtual supervisor trap handler base address.
0x240	HRW	vsscratch	Virtual supervisor scratch register.
0x241	HRW	vsepc	Virtual supervisor exception program counter.
0x242	HRW	vscause	Virtual supervisor trap cause.
0x243	HRW	vstval	Virtual supervisor trap value.
0x244	HRW	vsip	Virtual supervisor interrupt pending.
0x280	HRW	vsatp	Virtual supervisor address translation and protection.

Chapter 3. Machine-Level ISA, Version 1.13

This chapter describes the machine-level operations available in machine-mode (M-mode), which is the highest privilege mode in a RISC-V hart. M-mode is used for low-level access to a hardware platform and is the first mode entered at reset. M-mode can also be used to implement features that are too difficult or expensive to implement in hardware directly. The RISC-V machine-level ISA contains a common core that is extended depending on which other privilege levels are supported and other details of the hardware implementation.

3.1. Machine-Level CSRs

In addition to the machine-level CSRs described in this section, M-mode code can access all CSRs at lower privilege levels.

3.1.1. Machine ISA (`misa`) Register

The `misa` CSR is a **WARL** read-write register reporting the ISA supported by the hart. This register must be readable in any implementation, but a value of zero can be returned to indicate the `misa` register has not been implemented, requiring that CPU capabilities be determined through a separate non-standard mechanism.

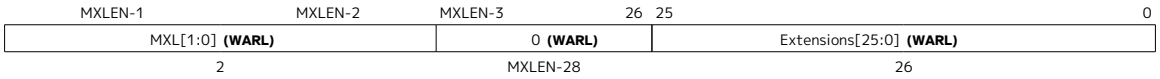


Figure 2. Machine ISA register (misa)

The MXL (Machine XLEN) field encodes the native base integer ISA width as shown in Table 9. The MXL field is read-only. If `misa` is nonzero, the MXL field indicates the effective XLEN in M-mode, a constant termed *MXLEN*. XLEN is never greater than MXLEN, but XLEN might be smaller than MXLEN in less-privileged modes.

Table 9. Encoding of MXL field in `misa`

MXL	XLEN
1	32
2	64
3	*Reserved*

The `misa` CSR is MXLEN bits wide.



The base width can be quickly ascertained using branches on the sign of the returned `misa` *value, and possibly a shift left by one and a second branch on the sign. These checks can be written in assembly code without knowing the register width (MXLEN) of the hart. The base width is given by MXLEN=2^{MXL+4}.*

The base width can also be found if `misa` *is zero, by placing the immediate 2 in a register, then shifting the register left by 31 bits. If zero, the hart is RV32, else it is RV64.*

The Extensions field encodes the presence of the standard extensions, with a single bit per letter of the alphabet (bit 0 encodes presence of extension "A" , bit 1 encodes presence of extension "B", through to bit 25 which encodes "Z"). The "I" bit will be set for the RV32I and RV64I base ISAs, and

the "E" bit will be set for RV32E and RV64E. The Extensions field is a **WARL** field that can contain writable bits where the implementation allows the supported ISA to be modified. At reset, the Extensions field shall contain the maximal set of supported extensions, and "I" shall be selected over "E" if both are available.

When a standard extension is disabled by clearing its bit in `misa`, the instructions and CSRs defined or modified by the extension revert to their defined or reserved behaviors as if the extension is not implemented.



For a given RISC-V execution environment, an instruction, extension, or other feature of the RISC-V ISA is ordinarily judged to be implemented or not by the observable execution behavior in that environment. For example, the F extension is said to be implemented for an execution environment if and only if the instructions that the RISC-V Unprivileged ISA defines for F execute as specified.

With this definition of implemented, disabling an extension by clearing its bit in `misa` *results in the extension being considered not implemented in M-mode. For example, setting* `misa`*.F=0 results in the F extension being not implemented for M-mode, because the F extension's instructions will not act as the Unprivileged ISA requires but may instead raise an illegal-instruction exception.*

Defining the term implemented based strictly on the observable behavior might conflict with other common understandings of the same word. In particular, although common usage may allow for the combination "implemented but disabled," in this document it is considered a contradiction of terms, because disabled implies execution will not behave as required for the feature to be considered implemented. In the same vein, "implemented and enabled" is redundant here; "implemented" suffices.

Table 10. Encoding of Extensions field in `misa`*. All bits that are reserved for future use must return zero when read.*

running the code. This register must be readable in any implementation. Hart IDs might not necessarily be numbered contiguously in a multiprocessor system, but at least one hart must have a hart ID of zero. Hart IDs must be unique within the execution environment.

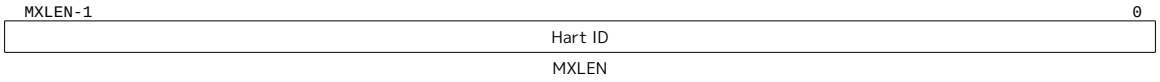


Figure 6. Hart ID (`mhartid`) register



In certain cases, we must ensure exactly one hart runs some code (e.g., at reset), and so require one hart to have a known hart ID of zero.

For efficiency, system implementers should aim to reduce the magnitude of the largest hart ID used in a system.

3.1.6. Machine Status (`mstatus` and `mstatush`) Registers

The `mstatus` register is an MXLEN-bit read/write register formatted as shown in Figure 7 for RV32 and Figure 8 for RV64. The `mstatus` register keeps track of and controls the hart's current operating state. A restricted view of `mstatus` appears as the `sstatus` register in the S-level ISA.

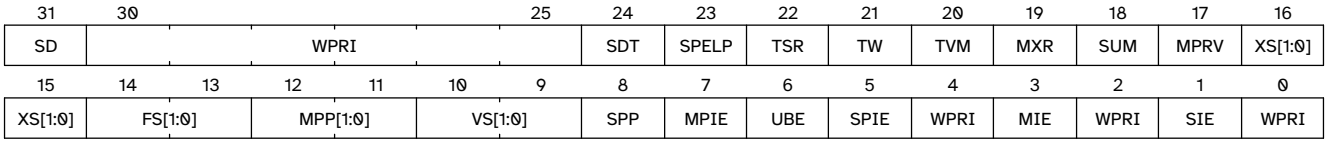


Figure 7. Machine-mode status (`mstatus`) register for RV32

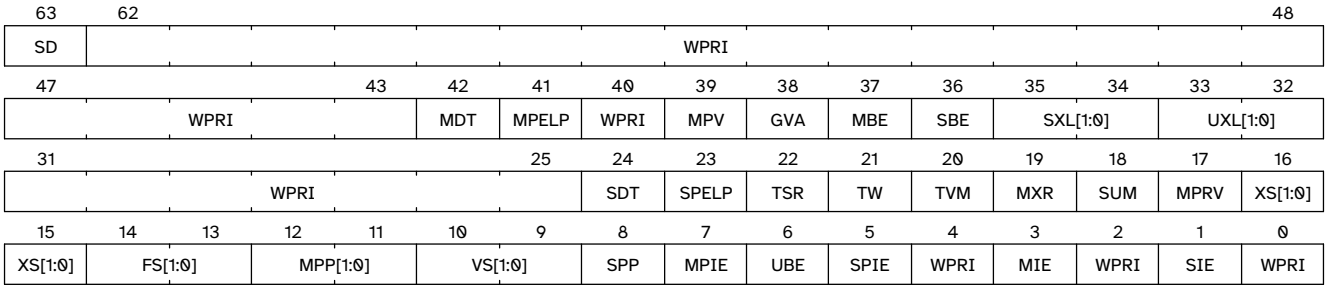


Figure 8. Machine-mode status (`mstatus`) register for RV64

For RV32 only, `mstatush` is a 32-bit read/write register formatted as shown in Figure 9. Bits 30:4 of `mstatush` generally contain the same fields found in bits 62:36 of `mstatus` for RV64. Fields SD, SXL, and UXL do not exist in `mstatush`.

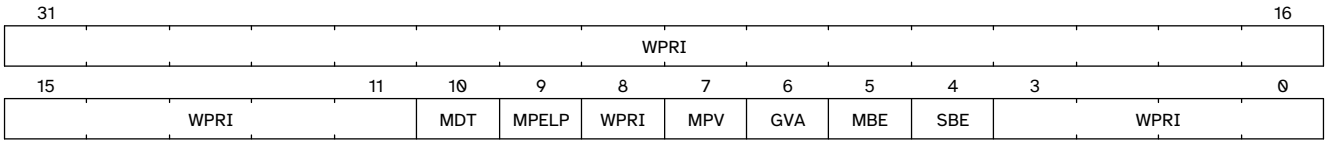


Figure 9. Additional machine-mode status (`mstatush`) register for RV32.

3.1.6.1. Privilege and Global Interrupt-Enable Stack in `mstatus` register

Global interrupt-enable bits, MIE and SIE, are provided for M-mode and S-mode respectively. These bits are primarily used to guarantee atomicity with respect to interrupt handlers in the current privilege mode.